

Ejercicio 3. Estado del arte: cinco artículos

Práctica 1: Operaciones Básicas sobre Lenguajes · Teoría de la Computación

Este documento reúne las cinco fichas solicitadas: tres textos asignados (Gribkoff, 2013; Luna-Benoso et al., 2022; y el trabajo original de Turing, 1936) y dos localizados (Davis et al., 2018; Vaandrager et al., 2022). Los dos últimos están publicados en un congreso con revisión por pares, cuentan con DOI, son posteriores a 2018 y son distintos de los asignados. El texto de Davis et al. aplica las expresiones regulares y los autómatas a un problema ajeno a la teoría (la seguridad de software). Para el Artículo 3 se eligió la opción recomendada, el trabajo de Turing.

Ficha 1. Gribkoff (2013): Applications of deterministic finite automata

1. Cita (APA 7). Gribkoff, E. (2013). *Applications of deterministic finite automata*

[Documento de curso, ECS 120]. University of California, Davis.

<https://www.cs.ucdavis.edu/~rogaway/classes/120/spring13/eric-dfa.pdf>

2. Problema que aborda. El curso ECS 120 pone el énfasis en la base teórica de los autómatas finitos deterministas (AFD). Este documento busca mostrar para qué sirven en sistemas reales que deben conservar un estado interno: máquinas expendedoras, comportamiento de personajes de videojuegos, protocolos de comunicación y búsqueda de texto (Gribkoff, 2013).

3. Método o propuesta. Primero fija la definición formal del AFD como una 5-tupla $(Q, \Sigma, \delta, q_0, F)$ y la de la máquina de Mealy como una 6-tupla $(Q, \Sigma, \Gamma, \delta, \omega, q_0)$. Después modela cuatro casos con diagramas de estados: (i) una máquina expendedora que cobra \$1.25 por refresco, cuyos estados representan el saldo acumulado y cuyo alfabeto es $\{\$0.25, \$1.00, \text{select}\}$; (ii) los cuatro comportamientos de los fantasmas de Pac-Man (vagar, perseguir, huir y volver a la base); (iii) una versión simplificada del protocolo TCP, vista como el ciclo de vida de una conexión; y (iv) el autocompletado de Apache Lucene mediante una máquina de Mealy.

4. Resultado principal. Es un texto expositivo y no reporta experimentos. Su conclusión es que los AFD, y sus extensiones con salida, describen de forma concisa cualquier sistema que deba recordar un estado. En el caso de Lucene, afirma que el transductor comparte los prefijos y sufijos comunes de los términos almacenados, lo que reduce la memoria y permite mantener todo el índice en memoria principal con búsquedas más rápidas. Esta última afirmación no se acompaña de datos.

5. Tema de la Unidad Temática I. Autómatas finitos: definición formal del AFD, función de transición, estados de aceptación y diagrama de transiciones; además, su extensión a máquinas con salida (Mealy).

6. Aportación al trabajo del curso. Ofrece un modelo de razonamiento para el Ejercicio 4: en la expendedora, cada estado representa una información del prefijo leído (el saldo acumulado), que es justamente lo que se pide documentar para cada autómata de JFLAP. También recuerda que un autómata puede modelar un sistema reactivo y no solamente reconocer un lenguaje.

Preguntas sobre el texto

a) Diferencia entre un AFD y una máquina de Mealy, y por qué el autocompletado de Lucene requiere la segunda. Un AFD es una 5-tupla con un conjunto F de estados de aceptación: al terminar de leer una cadena solo puede responder sí o no, según el estado final pertenezca o no a F , de modo que reconoce un lenguaje. Una máquina de Mealy sustituye F por un alfabeto de salida Γ y una función de salida $\omega : Q \times \Sigma \rightarrow \Gamma$. Produce una salida en cada transición, que depende del estado actual y del símbolo leído, y por eso no acepta ni rechaza: transforma una cadena de entrada en una cadena de salida, como una función de cadenas a cadenas (Gribkoff, 2013).

Lucene no solo necesita saber si un término pertenece al conjunto, que es lo que haría un AFD, sino obtener su posición en el arreglo ordenado de términos. En el ejemplo del documento, con el conjunto {mop, moth, pop, star, stop, top}, la palabra *stop* recorre arcos con salidas 3, vacía, 1 y vacía; la suma, 4, es su índice. Un AFD solo habría dicho que *stop* pertenece al conjunto. Además, la estructura comparte prefijos y sufijos entre términos, lo que reduce la memoria (Gribkoff, 2013). La máquina de Mealy es un tipo de transductor de estados finitos (FST).

b) La máquina expendedora y el conjunto de aceptación vacío. Una cadena w es aceptada si, al leerla desde q_0 , el autómata termina en un estado de F . Como $F \subseteq Q$ admite el conjunto vacío, declarar $F = \emptyset$ sigue siendo un AFD válido, pero ninguna cadena, ni siquiera la vacía, puede ser aceptada: el lenguaje reconocido es $L(M) = \emptyset$. La noción de “cadena aceptada” pierde entonces su utilidad, y el diagrama funciona como el modelo de un sistema reactivo: lo que importa es el estado alcanzado (el saldo) y las transiciones permitidas, por ejemplo que *select* tenga efecto solo desde los estados con al menos \$1.25 (Gribkoff, 2013). Si se quisiera tratar el modelo como un lenguaje, el de las secuencias de monedas que habilitan la compra, lo natural sería declarar como estados de aceptación los que el texto colorea de azul, es decir, los de saldo mayor o igual que \$1.25. Esta última precisión es una lectura propia del diagrama.

c) Afirmación que convendría verificar en una fuente arbitrada. La afirmación de que usar un transductor de estados finitos reduce de forma “drástica” la memoria del autocompletado de Lucene y permite guardar el índice completo en memoria, con búsquedas mucho más rápidas. Es una afirmación cuantitativa de desempeño, respaldada solo por un enlace a una entrada de blog (no arbitrada) y sin datos propios; además, el documento carece de bibliografía. Habría que contrastarla con literatura

arbitrada sobre diccionarios representados como autómatas acíclicos mínimos, como Daciuk et al. (2000), que describe cómo construir autómatas deterministas acíclicos mínimos a partir de un conjunto de cadenas, y buscar una evaluación experimental de las tasas de compresión. Una observación adicional: en el diagrama de TCP varias transiciones se etiquetan con pares entrada/salida (por ejemplo, SYN/SYN + ACK), lo que corresponde a una máquina de Mealy, aunque el texto lo presenta como un AFD.

Ficha 2. Luna-Benoso et al. (2022): detección de melanoma con un clasificador de autómatas celulares

1. Cita (APA 7). Luna-Benoso, B., Martínez-Perales, J. C., Cortés-Galicia, J., Flores-Carapia, R., y Silva-García, V. M. (2022). Melanoma detection in dermoscopic images using a cellular automata classifier. *Computers*, 11(1), Artículo 8.
<https://doi.org/10.3390/computers11010008>

2. Problema que aborda. El melanoma es el cáncer de piel más agresivo y mortal, y su detección temprana está estrechamente ligada a la supervivencia; además, es difícil de diagnosticar por su parecido con los nevos benignos. Los autores buscan un sistema de diagnóstico asistido por computadora (CAD) que detecte melanoma en imágenes dermatoscópicas.

3. Método o propuesta. El sistema tiene tres módulos. (i) *Segmentación* de la lesión con métodos del dominio espacial: filtro de mediana, binarización con el método de Otsu, eliminación del ruido de las esquinas, tres erosiones y tres dilataciones con la vecindad de Moore como elemento estructurante, y una operación *and* con la imagen a color. (ii) *Extracción de características* con descriptores de textura basados en 11 medidas estadísticas. (iii) *Clasificación* con un modelo asociativo basado en autómatas celulares (ACA), con una fase de aprendizaje a partir de un conjunto fundamental de pares de patrones y una fase de recuperación.

4. Resultado principal. Aplicado a imágenes de la base PH2 y comparado con otros modelos mediante exactitud, sensibilidad y especificidad, el método obtuvo 0.978, 0.944 y 0.987, respectivamente. Los autores concluyen que es más efectivo que otros métodos del estado del arte para esta tarea.

5. Tema de la Unidad Temática I. Concepto de autómata como modelo de cómputo con conjunto finito de estados y función de transición. El artículo sirve para contrastar el autómata finito con otro modelo basado en estados, el autómata celular.

6. Aportación al trabajo del curso. Muestra que un modelo definido por estados y una regla de transición puede usarse fuera del reconocimiento de lenguajes, y ayuda a precisar los límites del autómata finito (un único estado de control y una cadena de entrada) frente a otros modelos.

Preguntas sobre el texto

a) Autómata celular frente a autómata finito. Los autores definen un autómata celular como una tupla $(\mathcal{L}, S, N, f)$ en la que $\mathcal{L}$ es una retícula regular de celdas, S es un conjunto finito de estados, N es un conjunto de vecindades y $f : N \rightarrow S$ es la función de transición. Una configuración es una función $C_t : \mathcal{L} \rightarrow S$ que asigna un estado a cada celda en el instante t (Luna-Benoso et al., 2022). Para el AFD se usa la definición de Gribkoff (2013), con $\delta : Q \times \Sigma \rightarrow Q$.

Concepto	Autómata finito (AFD)	Autómata celular
Estado	Un solo elemento de Q , que resume la información relevante del prefijo leído.	Cada celda tiene un estado de S . El “estado global” es la configuración $C_t : \mathcal{L} \rightarrow S$.
Vecindario	No existe. La entrada es un símbolo de la cadena, leído en secuencia.	El conjunto $v(r)$ de celdas que la celda r “observa”. En el artículo se usa la vecindad de Moore en las operaciones morfológicas.
Regla de transición	$\delta(q, a)$: el siguiente estado depende del estado actual y del símbolo leído.	f : el siguiente estado de la celda r depende de los estados de las celdas de su vecindad, $C_{\{t+1\}}(r) = f(\{C_t(i) \mid i \in N(r)\})$, con la misma regla para todas las celdas.
Función	Reconoce un lenguaje: acepta si termina en un estado de F .	En su definición no hay alfabeto de entrada ni estados de aceptación; evoluciona una configuración. En el artículo se emplea como clasificador.

Semejanzas. Ambos modelos tienen un conjunto finito de estados, evolucionan en pasos discretos y su siguiente estado depende solo de la situación actual, no de la historia. Una celda individual puede verse como un autómata finito cuya entrada son los estados de sus vecinas. *Diferencias.* El AFD tiene un único estado global y lee una cadena secuencialmente; el autómata celular tiene un estado por celda, actualiza muchas celdas con una regla local y su información de entrada está distribuida en el espacio. El AFD termina con una decisión de aceptación; el autómata celular, tal como se define en el artículo, no la incluye.

b) Problema y elección del modelo. El problema es detectar melanoma en imágenes dermatoscópicas para apoyar el diagnóstico temprano. Los autores eligen el autómata celular porque es un modelo sencillo de implementar y requiere pocos recursos

computacionales en comparación con modelos de aprendizaje profundo como AlexNet o VGG16. Además, quieren llevar conceptos del campo de los autómatas celulares al aprendizaje supervisado (Luna-Benoso et al., 2022).

c) Resultado, conjunto de imágenes y métricas. El resultado es una exactitud de 0.978, una sensibilidad de 0.944 y una especificidad de 0.987. El conjunto de imágenes es la base de datos PH2 de imágenes dermatoscópicas, y las métricas son exactitud, sensibilidad y especificidad; el artículo también presenta una gráfica ROC que compara varios métodos de aprendizaje (Luna-Benoso et al., 2022).

Ficha 3. Turing (1936): On computable numbers, with an application to the Entscheidungsproblem

1. Cita (APA 7). Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1), 230-265.¹ <https://doi.org/10.1112/plms/s2-42.1.230>

2. Problema que aborda. El Entscheidungsproblem de Hilbert pregunta si existe un procedimiento mecánico que decida, para cualquier fórmula del cálculo funcional restringido, si es demostrable. Para responderlo hay que definir con precisión qué significa “procedimiento mecánico”, y Turing lo hace a través de los números computables: los reales cuyo desarrollo decimal puede escribir una máquina (Turing, 1936).

3. Método o propuesta. Turing compara a una persona que calcula con una máquina de un número finito de configuraciones (las *m-configuraciones*), con una cinta dividida en cuadros. En cada paso la máquina observa un cuadro y, según su configuración y el símbolo observado, imprime o borra un símbolo, se mueve un cuadro a la izquierda o a la derecha y cambia de configuración (secciones 1 y 2). A continuación: (i) enumera las máquinas mediante una descripción estándar y un número de descripción (sección 5); (ii) construye una máquina universal que simula a cualquier otra a partir de su descripción (secciones 6 y 7); (iii) aplica correctamente el argumento diagonal (sección 8); (iv) justifica que el modelo captura lo computable con tres tipos de argumentos (sección 9); y (v) reduce el Entscheidungsproblem a la pregunta de si una máquina imprime alguna vez un 0, construyendo para cada máquina M una fórmula $Un(M)$ (sección 11).

¹El registro del DOI fecha el volumen en 1937, pero la literatura suele citar el artículo como 1936. Ambas fechas circulan porque el trabajo se publicó por entregas dentro del volumen 42 de la serie 2 de los *Proceedings of the London Mathematical Society*, que abarca 1936 y 1937: fue recibido el 28 de mayo de 1936 y leído ante la Sociedad el 12 de noviembre de 1936 (Turing, 1936), y apareció en dos partes (pp. 230-240 y 241-265) a finales de 1936 (Christie's, s. f.). La corrección de Turing se publicó en 1937. Aquí se sigue la convención de 1936.

4. Resultado principal. Los números computables son enumerables, pero no existe una máquina que decida si una máquina dada es “libre de círculos”, es decir, si imprime una infinidad de cifras (sección 8); tampoco existe una que decida si una máquina imprime alguna vez un símbolo dado. De ello se deduce que el Entscheidungsproblem no tiene solución (sección 11). Grandes clases de números son computables: los reales algebraicos, π , e y los ceros de las funciones de Bessel (sección 10). En un apéndice se esboza la equivalencia entre computabilidad y la “calculabilidad efectiva” de Church. Una corrección posterior del propio Turing (1937) subsanó errores formales de la demostración de la sección 11.

5. Tema de la Unidad Temática I. Orígenes de la Teoría de la Computación (programa de Hilbert y Entscheidungsproblem), computabilidad y tesis de Church-Turing.

6. Aportación al trabajo del curso. Es el origen del modelo que ocupa el nivel superior de la jerarquía de Chomsky (las máquinas de Turing reconocen los lenguajes tipo 0) y permite situar al autómata finito como un modelo restringido: tiene control finito, pero no escribe sobre una cinta ilimitada. También fundamenta la idea de que algunas preguntas sobre programas no admiten un algoritmo general.

Preguntas sobre el texto

a) Qué resultado se demuestra y qué modelo se introduce. El modelo es la *máquina de cómputo* (hoy, máquina de Turing): control de estados finitos, cinta de cuadros, un cuadro observado y movimientos de un cuadro por paso. Una secuencia es computable si la calcula una máquina libre de círculos, y un número es computable si difiere en un entero de un número calculado por una de ellas. El resultado central es negativo: no hay un procedimiento general para decidir si una máquina es libre de círculos ni si imprimirá un símbolo dado, y de esa imposibilidad se obtiene que el Entscheidungsproblem no puede resolverse mecánicamente. La máquina universal es un resultado constructivo: una sola máquina puede calcular cualquier secuencia computable si se le entrega la descripción de la máquina que la produce (Turing, 1936).

b) La tesis de Church-Turing y por qué es una tesis. Enunciado: toda función efectivamente calculable, es decir, calculable por un procedimiento mecánico, finito y explícito, es computable por una máquina de Turing. La implicación inversa (toda función computable por una máquina de Turing es efectivamente calculable) se considera inmediata. Para las funciones de los enteros positivos, la versión de Church (efectivamente calculable implica recursiva) y la de Turing son equivalentes, porque Church, Kleene y Turing demostraron que los modelos coinciden (Copeland, 2012). Los modelos probados equivalentes son las máquinas de Turing, la λ -definibilidad de Church y Kleene, y las funciones recursivas; Turing esboza en su apéndice la equivalencia entre su computabilidad y la λ -definibilidad. El nombre “tesis de Church-Turing” se debe a Kleene y no aparece en los trabajos de 1936 (Copeland, 2012).

Se llama tesis y no teorema porque iguala un concepto formal (computable por una máquina de Turing) con uno informal e intuitivo (efectivamente calculable). Un teorema exige que ambos lados estén definidos formalmente. El propio Turing lo reconoce en la sección 9: todos los argumentos que puede ofrecer son, en el fondo, apelaciones a la intuición y, por eso, poco satisfactorios desde el punto de vista matemático. Propone tres tipos de argumentos: una apelación directa a la intuición, la demostración de la equivalencia de dos definiciones y ejemplos de grandes clases de números computables (Turing, 1936). La tesis podría refutarse exhibiendo una función efectivamente calculable que ninguna máquina de Turing calcule, pero hasta ahora toda función efectivamente calculable investigada ha resultado computable por una de ellas (Copeland, 2012).

c) Dificultad de lectura y cómo se resolvió. La principal dificultad fue que el vocabulario y la notación no coinciden con los del curso: Turing habla de *m-configuraciones* y no de estados, de *máquinas de cómputo* que imprimen indefinidamente cifras 0 y 1 (no de máquinas que se detienen ante una entrada), de máquinas “circulares” y “libres de círculos”, y usa letras góticas para las funciones de configuración abreviadas. Las tablas abreviadas y la descripción detallada de la máquina universal (secciones 4 a 7) son densas. Además, el artículo tuvo errores formales en la sección 11 que Turing corrigió en 1937. Se resolvió con una lectura por capas: primero las secciones 1 a 3 y 8 a 11, que contienen las ideas, y las secciones 4 a 7 solo para ver la arquitectura de la máquina universal; y construyendo una tabla de equivalencias entre sus términos y los del curso (*m*-configuración = estado, cuadro = celda de la cinta, libre de círculos = escribe una infinidad de cifras). Esta respuesta describe dificultades reales del texto y conviene ajustarla a la experiencia personal de lectura.

Ficha 4. Davis et al. (2018): el impacto de ReDoS en la práctica

1. Cita (APA 7). Davis, J. C., Coghlan, C. A., Servant, F., y Lee, D. (2018). The impact of regular expression denial of service (ReDoS) in practice: An empirical study at the ecosystem scale. En *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '18)* (pp. 246-256). Association for Computing Machinery. <https://doi.org/10.1145/3236024.3236027>

2. Problema que aborda. Muchos lenguajes de programación evalúan las expresiones regulares con motores de retroceso, cuyo tiempo en el peor caso puede ser polinomial o exponencial en la longitud de la entrada. Un atacante puede explotarlo para agotar la CPU de un servidor (ReDoS). No se sabía cuán común es el problema en el software real, cómo prevenirlo ni cómo repararlo.

3. Método o propuesta. Estudio empírico a escala de ecosistema. Extrajeron estáticamente, con árboles de sintaxis abstracta, las expresiones regulares de las bibliotecas centrales de Node.js y Python y de 448 402 módulos (más de la mitad) de los

registros npm y PyPI. Identificaron las expresiones de comportamiento superlineal con tres detectores previos (rxrx2, regex-static-analysis y rexploiter) y las validaron dinámicamente con entradas maliciosas formadas por prefijo, bombeo y sufijo, con un umbral de 10 segundos y hasta 85 615 repeticiones del bombeo. Clasificaron la gravedad ajustando curvas exponenciales y polinomiales. Después probaron tres antipatrones y estudiaron las estrategias de reparación en informes de vulnerabilidad y en 284 vulnerabilidades que ellos mismos notificaron a los mantenedores.

4. Resultado principal. Hallaron más de 4 000 expresiones regulares superlineales únicas, cerca de 300 de ellas exponenciales, en más de 10 000 módulos; alrededor del 1 % de las expresiones únicas era superlineal, y afectaba al 3 % de los módulos de npm y al 1 % de los de PyPI. Los antipatrones habituales aparecen en el 81-86 % de las expresiones superlineales, pero también en muchas seguras: son condiciones necesarias, no suficientes. De las 284 vulnerabilidades notificadas, 48 se repararon, y los desarrolladores prefirieron revisar la expresión (73 % de las nuevas correcciones) antes que truncar la entrada o reemplazarla por otro código. Concluyen que ReDoS es una vulnerabilidad común y no un caso marginal.

5. Tema de la Unidad Temática I. Autómatas finitos y expresiones regulares: el motor construye un AFN a partir de la expresión y lo simula, y el costo depende de cuántos estados explora. Se relaciona con la equivalencia entre AFN y AFD, porque los motores de tiempo lineal (como los de Rust y Go) evitan el retroceso con el algoritmo de Thompson. El texto menciona además que las referencias hacia atrás y los *lookaround* impiden un motor de tiempo lineal; el formalismo de la teoría no los incluye.

6. Aportación al trabajo del curso. Muestra una consecuencia práctica de la diferencia entre simular un AFN con retroceso y ejecutar un AFD, relevante para el Ejercicio 4, donde se diseñan autómatas deterministas y no deterministas. Además, la defensa que resultó más barata, limitar la longitud de la entrada, es análoga al límite de 200 000 cadenas que debe imponer la aplicación del Ejercicio 5 para controlar el crecimiento $|Σ|^n$.

Ficha 5. Vaandrager et al. (2022): aprendizaje activo de autómatas basado en la apartidad

1. Cita (APA 7). Vaandrager, F., Garhewal, B., Rot, J., y Wißmann, T. (2022). A new approach for active automata learning based on apartness. En D. Fisman y G. Rosu (Eds.), *Tools and algorithms for the construction and analysis of systems: 28th International Conference, TACAS 2022 (Lecture Notes in Computer Science, Vol. 13243, pp. 223-243)*. Springer. https://doi.org/10.1007/978-3-030-99524-9_12

2. Problema que aborda. El aprendizaje activo de autómatas consiste en inferir el AFD o la máquina de Mealy de un sistema desconocido haciendo preguntas a un “profesor”: de pertenencia (¿está w en el lenguaje?) y de equivalencia (¿es este modelo equivalente al sistema?). El algoritmo clásico L^* de Angluin (1987) y sus descendientes aproximan la

congruencia de Nerode por refinamiento; los autores buscan un enfoque más sencillo con la misma eficiencia.

3. Método o propuesta. El algoritmo L# se basa en establecer la *apartidad* (apartness), una forma constructiva de desigualdad: dos estados están aparte si existe una cadena de entrada, un testigo, que produce salidas distintas en ambos. En lugar de tablas de observación o árboles de discriminación, L# trabaja directamente sobre un *árbol de observación*, que es en sí una máquina de Mealy parcial, dividido en una base de estados ya identificados y una frontera. Aplica cuatro reglas hasta construir una hipótesis consistente con el árbol, procesa los contraejemplos con búsqueda binaria y puede incorporar secuencias distinguidoras adaptativas para mejorar el desempeño.

4. Resultado principal. Con n estados, k símbolos de entrada y un contraejemplo más largo de longitud m , L# aprende el modelo con $O(kn^2 + n \log m)$ consultas de salida y $O(kmn^2 + nm \log m)$ símbolos de entrada, la misma complejidad asintótica que los mejores algoritmos conocidos. En experimentos con modelos de SSH, TCP, TLS y tarjetas bancarias (el mayor, de 66 estados y 13 entradas), con un prototipo en Rust y comparado con TTT, ADT y RS de LearnLib, la variante con secuencias distinguidoras adaptativas resultó competitiva y entre las más rápidas en la fase de aprendizaje. Al contar también las pruebas de conformidad, todos los algoritmos quedan muy cerca. El prototipo no logró aprender un modelo de 3 410 estados porque el árbol de observación crece demasiado.

5. Tema de la Unidad Temática I. Definición formal del AFD y de la máquina de Mealy, lenguajes regulares, equivalencia de estados y distinguibilidad de estados.

6. Aportación al trabajo del curso. Da sentido formal a lo que se pide documentar en el Ejercicio 4: qué información del prefijo representa cada estado. Dos estados pueden distinguirse exactamente cuando existe una cadena que los separa, y esa idea es la base de la minimización de AFD y de la congruencia de Nerode. Además, retoma la máquina de Mealy del Artículo 1 y muestra su uso para modelar protocolos reales.

Cierre: tabla comparativa y comparación de los cinco textos

Texto	Arbitrado	Año	Modelo que usa	Campo de aplicación
1. Gribkoff	No (documento de curso)	2013	AFD y máquina de Mealy (transductor de estados finitos)	Máquinas expendedoras, IA de videojuegos, protocolos de red (TCP) y autocompletado de búsqueda (Lucene)
2. Luna-Benoso et al.	Sí (Computers)	2022	Autómata celular (clasificador asociativo)	Diagnóstico asistido por computadora: melanoma en imágenes dermatoscópicas
3. Turing (original)	Sí (Proc. London Math. Soc.)	1936	Máquina de Turing y máquina universal	Fundamentos de la computación y la lógica: computabilidad y Entscheidungsproblem
4. Davis et al.	Sí (ESEC/FSE)	2018	Expresiones regulares y AFN simulados por retroceso	Seguridad de software: ReDoS en módulos de npm y PyPI
5. Vaandrager et al.	Sí (TACAS)	2022	AFD y máquinas de Mealy; árbol de observación	Aprendizaje de modelos de protocolos (SSH, TCP, TLS) y pruebas de conformidad

Los cinco textos describen un sistema mediante estados finitos y reglas de transición (Gribkoff, Vaandrager et al., Davis et al.), la evolución de configuraciones (Luna-Benoso et al.) o un cálculo mecánico (Turing), y todos suponen que el comportamiento de un sistema puede resumirse en un conjunto acotado de situaciones. Difieren en rigor: Turing y Vaandrager et al. ofrecen definiciones formales, demostraciones y cotas de complejidad; Luna-Benoso et al. y Davis et al. se apoyan en evidencia empírica; Gribkoff es material didáctico sin arbitraje ni bibliografía, útil para intuir aplicaciones pero no como evidencia. También difieren en propósito: delimitar lo computable, enseñar, diagnosticar, medir un riesgo de seguridad e inferir modelos de sistemas reales, respectivamente.

Del conjunto surge un problema abierto: obtener modelos de estados finitos correctos y verificables a escala real. L# no pudo aprender un modelo de 3 410 estados porque el

árbol de observación se volvió demasiado grande; cerca del 1 % de las expresiones regulares únicas de npm y PyPI sigue siendo vulnerable, y la reparación automática que preserve el lenguaje reconocido está aún en desarrollo; y el clasificador celular se evaluó solo en una base de imágenes (PH2). Turing explica por qué no existe una solución general para programas arbitrarios, de modo que el avance depende de trabajar con modelos restringidos.

Referencias

- Copeland, B. J. (2012). The Church-Turing thesis. En E. N. Zalta (Ed.), *The Stanford Encyclopedia of Philosophy* (ed. verano 2012). Stanford University.
<https://plato.stanford.edu/archives/sum2012/entries/church-turing/>
- Christie's. (s. f.). *Alan Mathison Turing (1912-1954): "On computable numbers, with an application to the Entscheidungsproblem"* [Descripción bibliográfica de lote].
<https://onlineonly.christies.com.cn/s/shoulders-giants-making-modern-world/foundation-modern-digital-computing-28/70364>
- Daciuk, J., Mihov, S., Watson, B. W., y Watson, R. E. (2000). Incremental construction of minimal acyclic finite-state automata. *Computational Linguistics*, 26(1), 3-16.
<https://aclanthology.org/J00-1002/>
- Davis, J. C., Coghlan, C. A., Servant, F., y Lee, D. (2018). The impact of regular expression denial of service (ReDoS) in practice: An empirical study at the ecosystem scale. En *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '18)* (pp. 246-256). Association for Computing Machinery. <https://doi.org/10.1145/3236024.3236027>
- Gribkoff, E. (2013). *Applications of deterministic finite automata* [Documento de curso, ECS 120]. University of California, Davis.
<https://www.cs.ucdavis.edu/~rogaway/classes/120/spring13/eric-dfa.pdf>
- Luna-Benoso, B., Martínez-Perales, J. C., Cortés-Galicia, J., Flores-Carapia, R., y Silva-García, V. M. (2022). Melanoma detection in dermoscopic images using a cellular automata classifier. *Computers*, 11(1), Artículo 8.
<https://doi.org/10.3390/computers11010008>
- Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1), 230-265. <https://doi.org/10.1112/plms/s2-42.1.230>
- Turing, A. M. (1937). On computable numbers, with an application to the Entscheidungsproblem: A correction. *Proceedings of the London Mathematical Society*, s2-43(1), 544-546.

Vaandrager, F., Garhewal, B., Rot, J., y Wißmann, T. (2022). A new approach for active automata learning based on apartness. En D. Fisman y G. Rosu (Eds.), *Tools and algorithms for the construction and analysis of systems: 28th International Conference, TACAS 2022* (Lecture Notes in Computer Science, Vol. 13243, pp. 223-243). Springer. https://doi.org/10.1007/978-3-030-99524-9_12